

Self-validating specifications

Making domain knowledge
explicit and checkable.

Thesis

Domain specifications should express intent in a form that tools can validate and connect directly to implementation.

01 Why we wanted this

Beyond written instructions

02 What we can validate

Explicit domain semantics

03 A concrete example

Definitions, mistakes & feedback

Build a shared domain language

Our experiments included Spec Kit and domain-focused documentation workflows.

Matt Pocock's [grill-with-docs](#)

1 Interview

Question assumptions.
Explore concrete scenarios.

2 Clarify the language

Resolve ambiguous terms.
Check claims against the code.

3 Record what's agreed

Document vocabulary
and important decisions.

How can we turn that shared understanding
into enforceable constraints?

Describe the business first

A shared language for the business we're modelling.

Concept	Business meaning	Bicycle rental example
Domain	The area of business we're modelling	Bicycle rental
Entity	Identity persists as properties change	A particular bicycle
Value object	A value with no separate identity	A registration number
Policy	A business decision expressed through rules	Is this bicycle eligible for rental?

Make some expectations enforceable

The bicycle implements the rental-eligibility policy.



Is the policy
declared?

Is its implementation
present?

Does it target
the expected object?

Defined in Rust through types, semantic macros and file structure.

Specify the decision through examples

Policy: assess rental eligibility

Given rental status	And condition	Expected outcome
Available	Serviceable	Eligible
Rented	Serviceable	Unavailable status
Retired	Serviceable	Unavailable status
Available	Maintenance required	Maintenance required

Each test evaluates the policy and compares its result with the expected outcome.

Read the specification as a domain

Bicycle Entity in Rental Fleet

Properties

Identity	Bicycle ID
State	Rental status
Condition	Bicycle condition

Behavioural examples

Rental status	Condition	Expected outcome
Available	Serviceable	Eligible
Rented	Serviceable	Unavailable status
Retired	Serviceable	Unavailable status
Available	Maintenance required	Maintenance required

Decision

Assess rental eligibility

Possible outcomes
Eligible Unavailable status Maintenance required

Connect the policy contract to implementation

A consistent structure makes the expected parts easy to review.

```
bicycle/  
  entity.rs  
  assess_rental_eligibility/  
    policy.rs  
    outcome.rs  
    evaluate.rs
```

`policy.rs`

Declares the decision
and its return type.

`outcome.rs`

Defines the possible answers.

`evaluate.rs`

Implements the policy for Bicycle.

```
fn assess_rental_eligibility(&self) -> RentalEligibilityOutcome;
```

Validate at different levels

Each check catches a different kind of mistake.

Structure

Required parts exist and target the expected object.

Example failure

The policy evaluation implementation is missing.

Domain model

Declared identities and supported references are consistent.

Example failure

A referenced entity is missing from the model inventory.

Behaviour

The policy returns the expected outcomes for specified cases.

Example failure

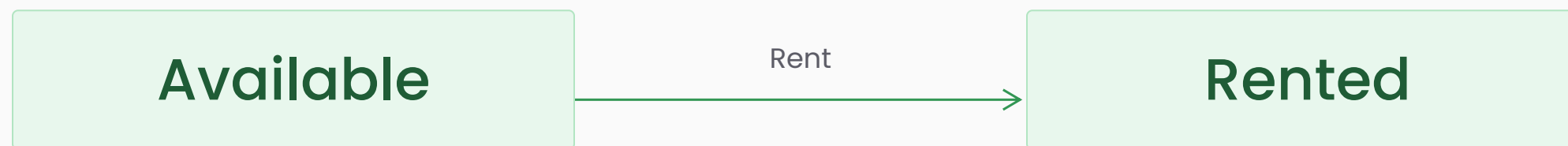
A bicycle needing maintenance is incorrectly eligible.

Structure checker diagnostic

policy directory requires ``evaluate.rs``

Evaluate a declared state rule

The declared transition participates in deciding whether renting is allowed.



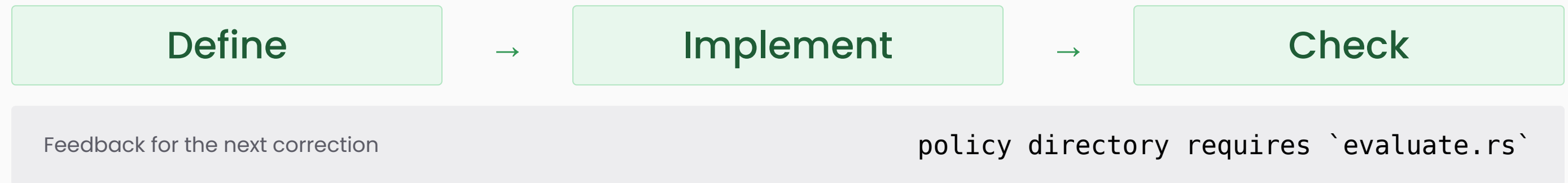
Declared in Rust

```
#[transition(id = "rent", label = "Rent")]  
#[edge(from = Available, to = Rented)]  
Rent,
```

Evaluate the current state

Current state	Attempt	Result
Available	Rent	Allowed
Rented	Rent	Rejected

Useful feedback for people and AI



Shared vocabulary

Domain concepts have explicit meaning.

Consistent structure

Developers can navigate and review the parts.

Concrete feedback

People and agents can act on reported violations.

Valid does not mean complete

A specification can pass its checks and still miss a business requirement.

Bicycle

Rental status

Available

Condition

Maintenance required

Did our specification
account for this?

People establish
the business meaning.

Tests cover
the examples we define.

Validation checks
the constraints we encode.

Make intent explicit enough to check

1 Define the domain explicitly

Concepts, relationships and decisions.

2 Validate at different levels

Structure, model consistency and behaviour.

3 Connect specification and implementation

Repeatable checks for developers and AI.

Which expectation in your specification could become a check?

Links

rostfrei on GitHub

github.com/elbLabs/rostfrei

Tom Last on LinkedIn

linkedin.com/in/tom-last-butnotleast

elbtech

elbtech.dev

Questions

Supporting concepts and technical details

B1 What do domain and bounded context mean?

B3 What is a policy versus an invariant, action or query?

B5 What do the macros add to ordinary code?

B7 Which tool checks which constraints?

B2 How do entities, value objects and aggregates differ?

B4 How do commands, events and transitions relate?

B6 How are tests connected to a policy?

B8 How is the domain exported and linked?

Domain, language and context

What do domain and bounded context mean?

Domain

The business area we are modelling.

Shared language

Agreed meanings used by developers and domain experts.

Bounded context

The boundary within which a model and its language apply.

A context boundary does not prescribe a deployment boundary.

Bike Rental

Rental fleet

Bicycle

Rental eligibility

Bicycle transfer

Identity, values and consistency

How do entities, value objects and aggregates differ?

Concept	What matters	Example
Entity	Identity persists as properties change	Bicycle
Value object	Meaning comes from its value	RegistrationNumber
Aggregate	Objects managed as a unit through a root	RentalFleetAggregate

Rental Fleet aggregate

RentalFleet root contains Bicycle entities.

Separate value-object example

RegistrationNumber is not a declared Bicycle field.

This example’s aggregate boundary is a modelling choice.

Decisions, constraints and operations

What is a policy versus an invariant, action or query?

Role	Question	Example
Policy	What decision follows from this situation?	Assess rental eligibility
Invariant	Does this state satisfy a required constraint?	Unique bicycle identities
Action	What operation can we perform?	Rent bicycle
Query	What information can we ask for?	Bicycle availability
Domain service	Where does behaviour involving multiple objects belong?	Bicycle transfer

Declaring an invariant does not automatically enforce it on every mutation.

Request, operation, fact and state change

How do commands, events and transitions relate?

Concept	Meaning	Example
Command	Request to perform an operation	RentBicycle
Action	Domain operation on the aggregate instance	RentBicycleAction
Event	Fact describing a successful change	BicycleRented
Transition	Declared edge between lifecycle states	Available → Rented

A command can be rejected. It is not already an event.

What is declared in Rust?

What do the macros add to ordinary code?

```
#[domain_policy(
    id = "assess-rental-eligibility",
    label = "Assess rental eligibility"
)]
pub(in crate::domain) trait RentalEligibilityPolicy {
    fn assess_rental_eligibility(&self)
        -> RentalEligibilityOutcome;
}
```

Macro

Semantic role, ID and label.

Trait

Callable contract and return type.

Implementation

Concrete target: Bicycle.

File structure

Placement and required parts.

The policy algorithm remains ordinary Rust implementation code.

Tests as executable examples

How are tests connected to a policy?

```
#[roster::domain_policy_test(
  <Bicycle as RentalEligibilityPolicy>::DESCRIPTOR
)]
fn returns_first_class_outcomes() {
  // Other cases omitted from this excerpt.
  assert_eq!(
    bicycle(
      BicycleStatus::Available,
      BicycleCondition::MaintenanceRequired,
    ).assess_rental_eligibility(),
    RentalEligibilityOutcome::MaintenanceRequired
  );
}
```

Linked subject

The policy descriptor.

Given

Available bicycle needing maintenance.

When

Evaluate rental eligibility.

Then

Expect MaintenanceRequired.

Validation responsibilities

Which tool checks which constraints?

Mechanism	Checks include	Does not establish
Compiler and macros	Types, traits and declaration syntax	Correct business meaning
Structure checker	Required files, placement and targets	Correct policy decisions
Compiled domain model	Duplicate IDs and supported references	Complete requirements
Behavioural tests	Expected results for specified cases	All untested cases
Export resolver	Supported explicit type relationships	A complete Rust call graph

Exporting declarations does not replace compilation, validation or tests.

From declarations to a readable model

How is the domain exported and linked?



Declared relationship	Resolved reference
Policy implementation	Domain object implementing the policy
Bicycle.status	Rental lifecycle type
Policy return type	Outcome type
CommandHandler<C>	Command C

Use the same source revision. Unresolved references are reported; method bodies are excluded.